Software Design Document

ADC Driver Module

Lab 03 – Microcontrollers and Embedded Systems

Module	ADC Driver
Version	1.0.0
Author(s)	J.M. Páez Sandoval, L.A. Lugo Muñiz, D. Lopez Moreno, C.A. Martínez Macías
Institution	CETYS Universidad — Facultad de Ingeniería
Date	22/05/2026

1. Introduction

1.1 Purpose

This document describes the design and architecture of the adc_driver module. It covers the handle-less configuration API, single-channel regular conversion, resolution and alignment control, sample time configuration, software-triggered conversion start, EOC polling with overrun detection, and data read.

1.2 Scope

The adc_driver is a low-level bare-metal module for STM32F4xx ADC peripherals. It operates in single-conversion software-triggered mode (no DMA, no interrupts, no scan mode). It supports 12/10/8/6-bit resolution, right/left alignment, all eight sample time settings (3–480 cycles), and channels 0–18. The module is pointer-based: the caller passes an ADC_TypeDef* directly, enabling use with ADC1, ADC2, or ADC3.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
ADC	Analog-to-Digital Converter — converts voltage to digital value
EOC	End-Of-Conversion flag (SR bit 1) — set by hardware when DR is ready
OVR	Overrun flag (SR bit 5) — set when a new conversion completes before DR is read
CR1	ADC Control Register 1 — resolution (RES), scan mode (SCAN)
CR2	ADC Control Register 2 — ADON, CONT, ALIGN, EOCS, SWSTART
SWSTART	Software Start bit (CR2 bit 30) — triggers a regular conversion

Term	Definition
EOCS	End-Of-Conversion Selection (CR2 bit 10) — EOC after each conversion
ALIGN	Data alignment bit (CR2 bit 11) — 0=right, 1=left in 16-bit DR
ADON	ADC On/Off bit (CR2 bit 0) — powers the ADC analog section
CONT	Continuous mode bit (CR2 bit 1) — 0=single, 1=continuous
SQR1	Regular Sequence Register 1 — L[23:20] = sequence length – 1
SQR3	Regular Sequence Register 3 — SQ1[4:0] = first channel in sequence
SMPR1	Sample Time Register 1 — 3-bit fields for channels 10–18
SMPR2	Sample Time Register 2 — 3-bit fields for channels 0–9
DR	Data Register — 16-bit; reading clears SR.EOC
RES	Resolution bits [25:24] in CR1 — 00=12b, 01=10b, 10=8b, 11=6b
ADC_Config_t	Configuration struct: resolution, align, channel, sampleTime
ADC_Status_t	Return type: ADC_OK=0, negative values = error codes

1.4 Requirements

All requirements are allocated in Docs/SRS/ADC_Driver_SRS.md (provided by instructor without modification). Functional requirements FR-1 to FR-8 and non-functional requirements NFR-1 to NFR-3 are addressed.

2. System Architecture

2.1 High-Level Design

The adc_driver sits directly above the ADC hardware registers. GPIO pin configuration (analog mode) must be performed by the caller before adc_init(). APB2 clock for the ADC must also be enabled by the caller. The driver performs no clock gating.

Layer	Modules
Application	main.c / sensor.c — calls adc_init, adc_enable, adc_start, adc_poll_eoc, adc_read
ADC Driver	All ADC register operations: config, power, trigger, poll, read
Hardware	STM32F411RE ADC1 (APB2). Analog input on GPIO pin configured as ANALOG mode.

2.2 Module Interfaces

Public API declared in adc_driver.h. Dependency: stm32f411xe.h. No HAL, no gpio_driver, no timer. The caller must: (1) enable RCC->APB2ENR.ADC1EN, (2) configure the GPIO pin as ANALOG mode, before calling adc_init().

3. Detailed Design

3.1 Class Diagram

Figure 1. Class Diagram — adc_driver module

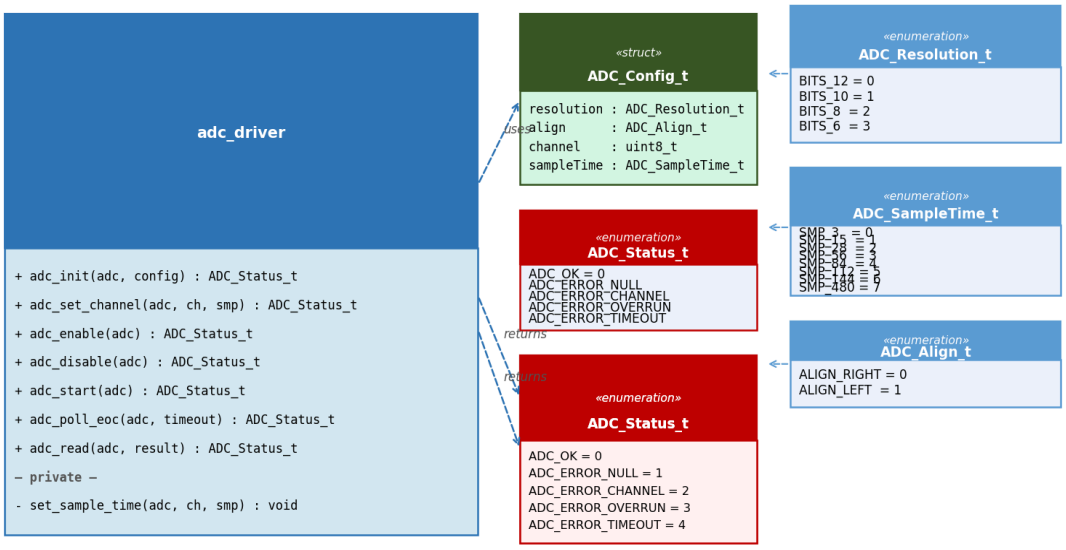

Figure 1. Class Diagram — adc_driver, ADC_Config_t, enumerations

3.2 Module Behavior

Figure 2. Sequence Diagram — adc_init() + adc_enable()

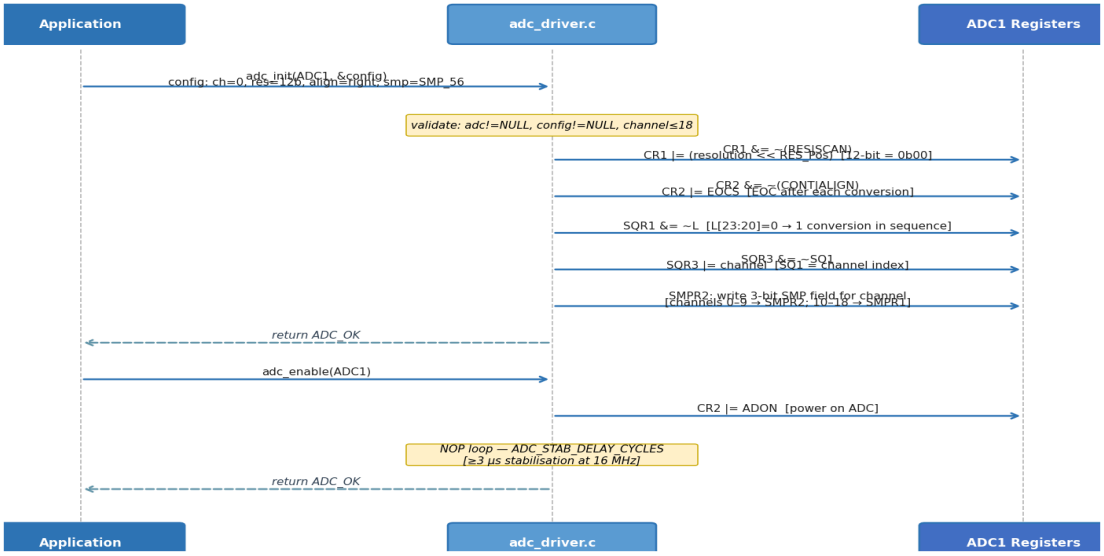

Figure 2. Sequence Diagram — adc_init() CR1/CR2/SQR/SMPR write, adc_enable() ADON + stabilisation delay

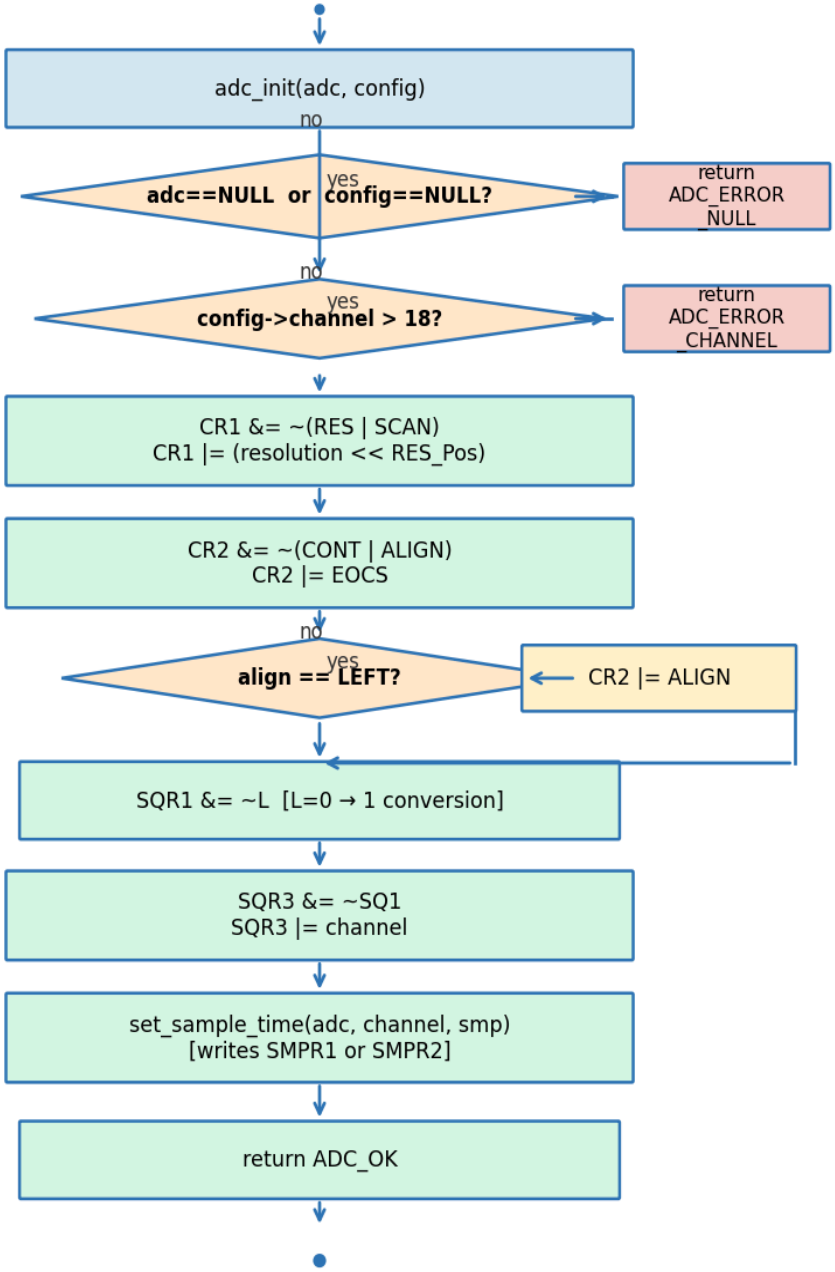

Figure 3. Activity Diagram — adc_init(): NULL/channel checks, register configuration, alignment branch

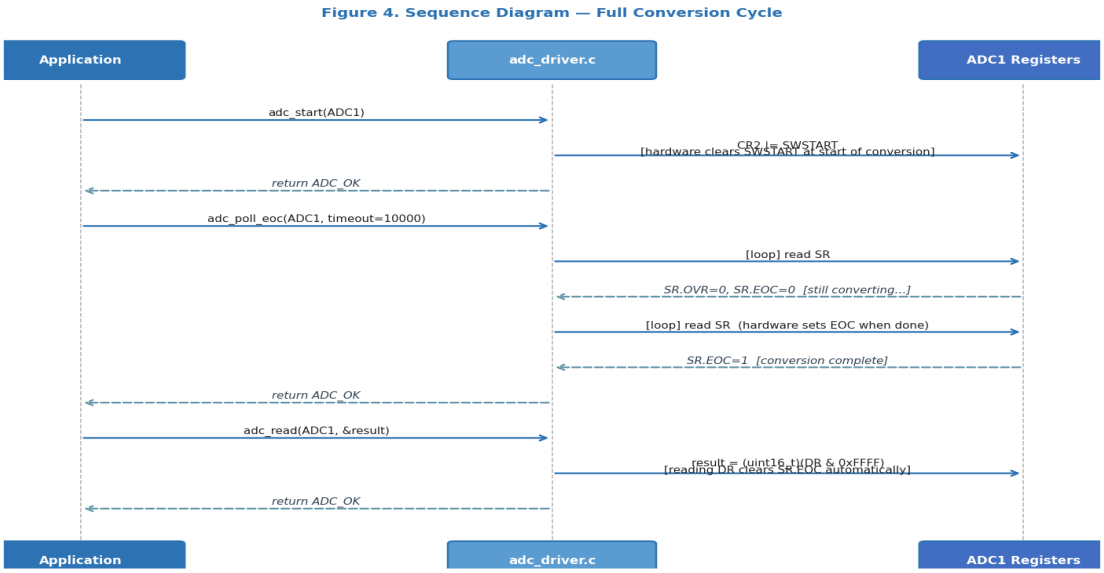

Figure 4. Sequence Diagram — Full conversion cycle: adc_start() → adc_poll_eoc() → adc_read()

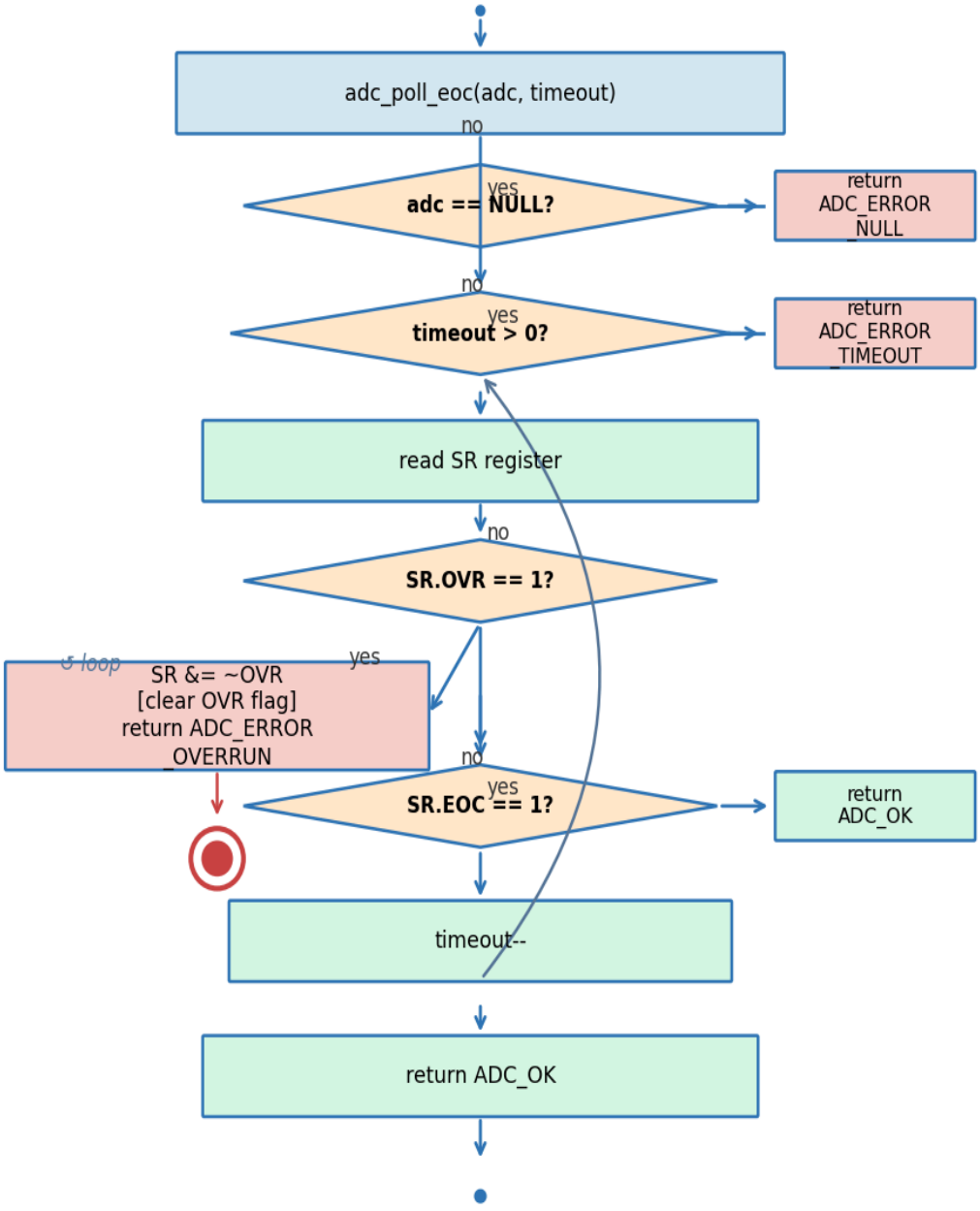

Figure 5. Activity Diagram — adc_poll_eoc(): OVR detection, EOC polling, timeout decrement loop

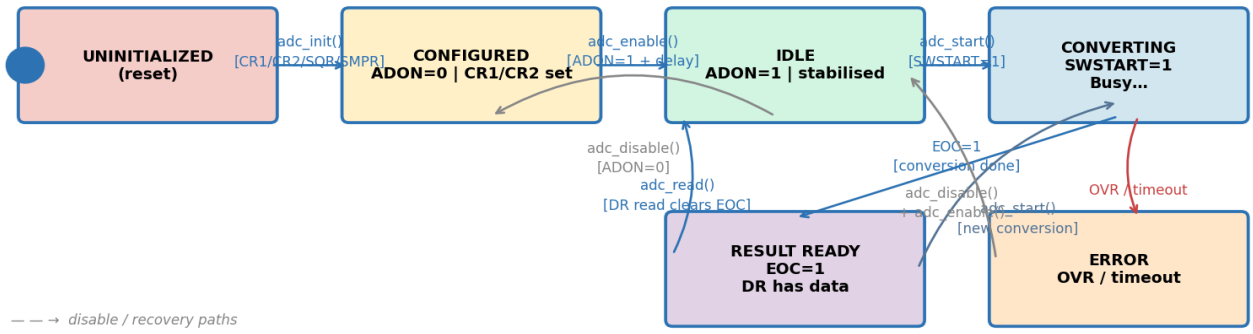

Figure 6. State Diagram — ADC Peripheral States

3.3 ADC Driver Module Description

adc_init() validates the ADC pointer and config struct, then configures four register groups. CR1: clears RES[25:24] and SCAN, writes resolution. CR2: clears CONT and ALIGN, sets EOCS=1 (EOC after each conversion), optionally sets ALIGN for left alignment. SQR1: clears L[23:20] to program a sequence of exactly one conversion. SQR3: writes the channel index into SQ1[4:0]. Finally calls adc_set_channel() to write the 3-bit sample time into SMPR1 or SMPR2 depending on the channel number.

adc_enable() sets CR2.ADON and executes a NOP loop for at least ADC_STAB_DELAY_CYCLES cycles (~3 µs at 16 MHz) to allow the ADC analog section to stabilise before the first conversion is triggered.

adc_start() sets CR2.SWSTART to trigger a software-initiated regular conversion. Hardware clears SWSTART automatically at the start of conversion.

adc_poll_eoc() polls SR in a timeout loop. On each iteration it first checks SR.OVR (a previous result was overwritten before DR was read) and returns ADC_ERROR_OVERRUN immediately after clearing the flag. Then checks SR.EOC — if set, returns ADC_OK. Decrements timeout on each iteration; returns ADC_ERROR_TIMEOUT if the counter reaches zero.

adc_read() reads DR as a 16-bit value masked with 0xFFFF. Reading DR clears SR.EOC automatically (hardware behaviour). For right-aligned 12-bit results the valid data is in bits [11:0].

set_sample_time() is a private helper that writes the 3-bit SMP field for a given channel. Channels 0–9 map to SMPR2 (field at bits [ch×3+2 : ch×3]); channels 10–18 map to SMPR1 (field at bits [(ch-10)×3+2 : (ch-10)×3]).

4. Application Programming Interfaces

4.1 Data Types and Macros

Type	Fields / Values	Notes
ADC_Config_t	resolution: ADC_Resolution_talign: ADC_Align_tchannel: uint8_t (0–18)sampleTime: ADC_SampleTime_t	Configuration struct passed by const pointer to adc_init()
ADC_Resolution_t	BITS_12=0, BITS_10=1BITS_8=2, BITS_6=3	Written to CR1.RES[25:24]
ADC_Align_t	ALIGN_RIGHT=0ALIGN_LEFT=1	Controls CR2.ALIGN bit
ADC_SampleTime_t	SMP_3=0, SMP_15=1, SMP_28=2SMP_56=3, SMP_84=4, SMP_112=5SMP_144=6, SMP_480=7	3-bit field in SMPR1/SMPR2
ADC_Status_t	ADC_OK=0, ADC_ERROR_NULL=1ADC_ERROR_CHANNEL=2 ADC_ERROR_OVERRUN=3ADC_ERROR_TIMEOUT=4	Return type for all public functions
ADC_MAX_CHANNEL	18U	Maximum valid channel index
ADC_STAB_DELAY_CYCLES	≥ 48U (3 µs @ 16 MHz)	NOP loop count in adc_enable()

4.2 Error Codes

Code	Value	Definition
ADC_OK	0	Operation completed successfully
ADC_ERROR_NULL	1	NULL pointer passed as adc or config/result
ADC_ERROR_CHANNEL	2	channel > ADC_MAX_CHANNEL (18)
ADC_ERROR_OVERRUN	3	SR.OVR was set — previous DR not read before new conversion finished
ADC_ERROR_TIMEOUT	4	timeout counter reached zero before SR.EOC was set

4.3 Function Definitions

adc_init

Function Name	adc_init
Syntax	ADC_Status_t adc_init(ADC_TypeDef *adc, const ADC_Config_t *config)

Parameters (in)	adc — ADC peripheral pointer (ADC1/ADC2/ADC3)config — const pointer to configuration struct
Return value	ADC_OK, ADC_ERROR_NULL, or ADC_ERROR_CHANNEL
Description	Configures CR1 (resolution, no scan), CR2 (single, EOCS, alignment), SQR1 (L=0), SQR3 (SQ1=channel), SMPR1/SMPR2 (sample time). Does not power on the ADC.
Constraints	Caller must enable APB2 ADC clock and configure GPIO pin as ANALOG before calling.
Available via	adc_driver.h

adc_set_channel

Function Name	adc_set_channel
Syntax	ADC_Status_t adc_set_channel(ADC_TypeDef *adc, uint8_t channel, ADC_SampleTime_t smp)
Parameters (in)	adc, channel (0–18), smp — sample time enum
Return value	ADC_OK, ADC_ERROR_NULL, or ADC_ERROR_CHANNEL
Description	Updates SQR3.SQ1 and the corresponding SMPR1/SMPR2 field. Allows channel change without full reinit.
Constraints	ADC must be disabled (ADON=0) when changing channel to avoid mid-conversion changes.
Available via	adc_driver.h

adc_enable

Function Name	adc_enable
Syntax	ADC_Status_t adc_enable(ADC_TypeDef *adc)
Parameters (in)	adc — ADC peripheral pointer
Return value	ADC_OK or ADC_ERROR_NULL
Description	Sets CR2.ADON then executes a NOP loop for ADC_STAB_DELAY_CYCLES to allow analog section stabilisation before the first SWSTART.
Constraints	Must be called after adc_init(). Satisfies FR-4.
Available via	adc_driver.h

adc_disable

Function Name	adc_disable
Syntax	ADC_Status_t adc_disable(ADC_TypeDef *adc)
Parameters (in)	adc — ADC peripheral pointer
Return value	ADC_OK or ADC_ERROR_NULL
Description	Clears CR2.ADON to power off the ADC analog section. Satisfies FR-5.

Constraints	Any ongoing conversion is aborted.
Available via	adc_driver.h

adc_start

Function Name	adc_start
Syntax	ADC_Status_t adc_start(ADC_TypeDef *adc)
Parameters (in)	adc — ADC peripheral pointer
Return value	ADC_OK or ADC_ERROR_NULL
Description	Sets CR2.SWSTART to software-trigger a regular conversion. Hardware clears SWSTART at the start of conversion. Satisfies FR-6.
Constraints	ADC must be enabled (ADON=1). adc_poll_eoc() must be called before the next adc_start() to avoid OVR.
Available via	adc_driver.h

adc_poll_eoc

Function Name	adc_poll_eoc
Syntax	ADC_Status_t adc_poll_eoc(ADC_TypeDef *adc, uint32_t timeout)
Parameters (in)	adc — ADC pointertimeout — max polling iterations (e.g. 10000)
Return value	ADC_OK, ADC_ERROR_NULL, ADC_ERROR_OVERRUN, or ADC_ERROR_TIMEOUT
Description	Polls SR in a loop: checks OVR first (clears flag, returns overrun error), then checks EOC (returns OK). Decrements timeout; returns timeout error at zero. Satisfies FR-7.
Constraints	Blocking (CPU busy-wait). timeout=0 returns ADC_ERROR_TIMEOUT immediately.
Available via	adc_driver.h

adc_read

Function Name	adc_read
Syntax	ADC_Status_t adc_read(ADC_TypeDef *adc, uint16_t *result)
Parameters (in)	adc — ADC pointerresult — pointer to destination uint16_t
Return value	ADC_OK or ADC_ERROR_NULL
Description	Reads DR & 0xFFFF into *result. Reading DR clears SR.EOC automatically. For 12-bit right-aligned, valid data is in bits [11:0]. Satisfies FR-8.
Constraints	Must be called after adc_poll_eoc() returns ADC_OK to ensure valid data.
Available via	adc_driver.h

4.4 Internal Function Definitions

set_sample_time (static)

Function Name	set_sample_time
Syntax	static void set_sample_time(ADC_TypeDef *adc, uint8_t channel, ADC_SampleTime_t smp)
Parameters (in)	adc, channel (0–18), smp — 3-bit sample time value
Return value	void
Description	Channels 0–9: writes 3-bit SMP field in SMPR2 at bits [(ch×3+2):(ch×3)]. Channels 10–18: writes in SMPR1 at bits [((ch-10)×3+2):((ch-10)×3)]. Uses read-modify-write.
Constraints	Not accessible outside adc_driver.c. Called by adc_init() and adc_set_channel().

5. Future Enhancements

• Add DMA-based continuous mode: enable CR2.CONT, configure DMA1 Stream0, and provide a callback for buffer-full notification.

• Add multi-channel scan mode: configure L[23:20] > 0 in SQR1 and populate SQR2/SQR3 with channel sequences.

• Implement interrupt-driven EOC: enable CR1.EOCIE and provide an IRQ handler with a completion callback.

• Add adc_calibrate() to perform the STM32F4 ADC factory calibration procedure for improved accuracy.